Assignment 3

1. Explain Different Modelling Styles (Data Flow, Behavioral, and Structural) in VHDL for Combinational and Sequential Circuits. Also, mention the major differences between them (any six).

VHDL supports three different modelling styles to describe digital circuits: Data Flow, Behavioral, and Structural modelling. These modelling styles can be used to design both combinational and sequential circuits.

(a) Data Flow Modelling:
In data flow modelling, the circuit is described using concurrent signal assignment statements. It represents how data moves from inputs to outputs using logical expressions. This modelling style mainly uses operators such as AND, OR, NOT, and XOR. It is mostly suitable for combinational circuits.

Example: Y <= A and B.

(b) Behavioral Modelling:
Behavioral modelling describes the behavior or functionality of a circuit rather than its internal structure. It uses process blocks along with control statements such as IF, CASE, and LOOP. This modelling style is suitable for both combinational and sequential circuits. Clock signals are used to model sequential circuits like flip-flops and counters.

(c) Structural Modelling:
Structural modelling represents the circuit as a collection of interconnected components. It is similar to a block diagram representation of the hardware. Each component is instantiated and connected using signals. This modelling style is suitable for both combinational and sequential circuits and provides a clear view of the hardware structure.

Major Differences between Modelling Styles (Any Six):

1. Data flow modelling focuses on data movement, while behavioral modelling focuses on functionality, and structural modelling focuses on hardware structure.
2. Data flow uses concurrent assignments, behavioral uses process blocks, and structural uses component instantiation.
3. Data flow and behavioral modelling provide less hardware detail compared to structural modelling.
4. Behavioral modelling supports sequential statements like IF and CASE, while data flow and structural modelling do not.
5. Structural modelling is more complex compared to data flow and behavioral modelling.
6. Behavioral modelling is most flexible for complex designs.

2. What are the Features of VHDL Programming?

VHDL programming has several important features that make it suitable for digital system design.

1. VHDL is technology independent and can be used for different hardware platforms.
2. It supports multiple levels of abstraction such as behavioral, data flow, and structural modelling.
3. VHDL supports concurrent as well as sequential execution of statements.
4. It is a strongly typed language, which helps in reducing design errors.
5. VHDL supports modular design using libraries, packages, and subprograms.
6. It supports both simulation and synthesis of digital circuits.

Available on notes.arpanadhikari7.com.np

7. Timing and delay specifications can be included in VHDL designs.
8. VHDL designs are portable across different tools and vendors.

3. What are the Advantages of VHDL Programming?

The advantages of VHDL programming are as follows:

1. Errors can be detected early through simulation before hardware implementation.
2. VHDL improves design reliability due to its strong typing and structured nature.
3. Design modules can be reused in different projects.
4. It supports the design of large and complex digital systems.
5. VHDL naturally supports parallel processing.
6. The code serves as proper documentation of the design.
7. It is independent of hardware vendors.
8. VHDL designs are easy to modify and maintain.

4. What are the Different Data Types Present in VHDL?

a) Scalar Data Types

- **Integer** – Whole numbers
- **Real** – Floating point numbers
- **Boolean** – TRUE or FALSE
- **Bit** – '0' or '1'

b) Enumerated Data Types

- **std_logic** – '0', '1', 'Z', 'X', etc.

c) Array Data Types

- **Bit_vector**
- **std_logic_vector**
- Used to represent buses and multi-bit signals.

notes.arpanadhikari7.com.np

d) Composite Data Types

- **Array**
- **Record**

e) Physical Data Types

- Used for time and delay representation.
- Example: time, ns, ms

f) Access Data Types

- Similar to pointers.

Available on notes.arpanadhikari7.com.np

5. Explain the Roles of Libraries, Subprograms, and Packages in VHDL.

Answer:

a) Libraries

- Libraries store **compiled VHDL design units**.
- They help organize large designs.
- Example: IEEE library.
- Used to access predefined packages.

b) Packages

- Packages contain **type definitions, constants, functions, and procedures**.
- They promote **code reusability and standardization**.
- Example: std_logic_1164 package.
- Declared once and used across multiple design units.

c) Subprograms

- Subprograms include **Functions and Procedures**.
- Functions return a single value.
- Procedures may have multiple outputs.
- Used to simplify and modularize code.

Role Summary:

- **Libraries** organize design units.
- **Packages** share common declarations.
- **Subprograms** reduce code repetition and improve readability.

6. What is the difference between std_logic and bit data types?

In VHDL, `bit` and `std_logic` are used to represent binary values, but they differ in terms of functionality and practical usage.

Bit Data Type:
The `bit` data type is a basic data type that can represent only two logic values: '0' and '1'. It is mainly used in simple and theoretical designs. It does not support unknown, uninitialized, or high-impedance states, which limits its use in real hardware modeling.

std_logic Data Type:
The `std_logic` data type is defined in the IEEE `std_logic_1164` package. It supports multiple logic states such as '0', '1', 'X', 'Z', 'U', 'W', 'L', 'H', and '-'. This allows accurate representation of real digital hardware conditions like bus contention and uninitialized signals.

Available on notes.arpanadhikari7.com.np

Differences between bit and std_logic:

1. `bit` supports only two values (0 and 1), whereas `std_logic` supports nine logic values.
2. `bit` cannot represent unknown or high-impedance states, but `std_logic` can.
3. `bit` does not support signal resolution, while `std_logic` supports resolved signals.
4. `bit` is rarely used in practical designs, whereas `std_logic` is industry standard.
5. `std_logic` provides better simulation accuracy than `bit`.
6. `std_logic` is suitable for modeling real hardware, while `bit` is limited.

7. Explain the significance of the std_logic_vector data type.

The `std_logic_vector` data type is an array type composed of multiple `std_logic` elements. It is one of the most important data types in VHDL.

Significance of std_logic_vector:

The `std_logic_vector` is used to represent multi-bit digital signals such as data buses, address lines, registers, and binary numbers. Each bit in the vector can independently take any valid `std_logic` value, making it suitable for accurate digital system modeling.

This data type is widely accepted in industry and supported by all major simulation and synthesis tools. It is also compatible with arithmetic and logical operations when used with appropriate libraries.

Key Points:

1. Represents multi-bit signals and buses.
2. Supports unknown and high-impedance states.
3. Improves accuracy of hardware simulation.
4. Enables scalable and modular design.
5. Widely supported by synthesis tools.
6. Essential for complex digital systems.

8. What are the different operators in VHDL? Provide examples of each type.

VHDL provides various operators to perform operations on data. These operators are classified based on their functionality.

Arithmetic Operators:
Used for mathematical calculations.
Examples: +, -, *, /, mod

Relational Operators:
Used to compare values and return Boolean results.
Examples: =, /=, <, >, <=, >=

Logical Operators:
Used to perform logical operations on signals.
Examples: and, or, not, nand, nor, xor, xnor

Available on notes.arpanadhikari7.com.np

Shift Operators:
Used to shift or rotate bits in vectors.
Examples: sll, srl, sla, sra, rol, ror

Concatenation Operator:
Used to combine two or more signals.
Example: &

Assignment Operators:
Used to assign values.
Signal assignment: <=
Variable assignment: :=

9. Explain how typecasting is done in VHDL with an example.

Typecasting in VHDL is the process of converting one data type into another. Since VHDL is a strongly typed language, explicit type conversion is mandatory.

Typecasting is mainly required when performing arithmetic operations or when assigning values between incompatible data types such as integer and `std_logic_vector`.

Typecasting is performed using standard conversion functions provided by libraries like `numeric_std`.

Common Type Conversions:

1. Integer to std_logic_vector
2. std_logic_vector to integer
3. std_logic_vector to signed or unsigned

Explanation:
For example, an integer value can be converted into a vector using a conversion function, and a vector can be converted back into an integer for arithmetic processing.

Typecasting ensures correctness, avoids compilation errors, and improves code clarity.

notes.arpanadhikari7.com.np

10. What is the purpose of an entity in VHDL?

An entity in VHDL defines the external interface of a design. It specifies how a digital circuit communicates with the outside world.

The entity declares the name of the design, input and output ports, their data types, and directions. It does not describe the internal working of the circuit.

Purpose of an Entity:

1. Defines the input and output ports of a design.
2. Specifies signal directions such as IN, OUT, INOUT, and BUFFER.
3. Provides a clear interface for module interaction.
4. Supports modular and hierarchical design.

Available on notes.arpanadhikari7.com.np

5. Allows reuse of the same entity with different architectures.
6. Helps in simulation and testbench development.

The entity works together with the architecture, where the actual behavior or structure of the design is implemented.

11. Explain the concept of architecture in VHDL and its types.

In VHDL, an architecture defines the **internal implementation** of an entity. While the entity specifies the external interface of a design, the architecture describes **how the circuit works internally**.

An entity can have **multiple architectures**, each representing a different way of implementing the same interface. This allows designers to choose different design styles for simulation or synthesis.

Concept of Architecture:

- It contains the actual logic of the design.
- It may include signals, processes, concurrent statements, and component instantiations.
- Architecture is always associated with an entity.

Types of Architecture in VHDL:

1. Behavioral Architecture
   Describes the behavior or functionality of the circuit using process blocks and sequential statements such as IF, CASE, and loops.
2. Data Flow Architecture
   Describes how data flows from inputs to outputs using concurrent signal assignment statements.
3. Structural Architecture
   Describes the circuit as an interconnection of components or modules.

Thus, architecture provides flexibility and supports different abstraction levels in VHDL design.

12. How are signals and ports used in an entity declaration?

In VHDL, an entity declaration defines the **external interface** of a design using ports. Ports communicate with the outside world, while signals are used internally to connect components or processes.

Ports in Entity Declaration:

- Ports are declared inside the entity.
- Each port has a name, direction, and data type.
- Port directions include IN, OUT, INOUT, and BUFFER.

Signals:

- Signals represent connections within the architecture.
- They are used to transfer data between concurrent statements or components.
- Signals are declared inside the architecture, not inside the entity.

Role Summary:

Available on notes.arpanadhikari7.com.np

- Ports are used for external communication.
- Signals are used for internal communication.
- Together, they enable data flow between the entity and its internal logic.

13. Differentiate between concurrent and sequential statements in VHDL.

VHDL supports both concurrent and sequential statements to model parallel hardware behavior.

Concurrent Statements:

- Executed simultaneously.
- Written outside process blocks.
- Used mainly in data flow and structural modeling.
- Example: concurrent signal assignment.

Sequential Statements:

- Executed one after another.
- Written inside process blocks.
- Used mainly in behavioral modeling.
- Example: IF, CASE, FOR loop.

Differences:

1. Concurrent statements execute in parallel, sequential statements execute step by step.
2. Concurrent statements are written outside processes, sequential statements are inside processes.
3. Concurrent statements represent hardware parallelism directly.
4. Sequential statements follow software-like execution order.
5. Concurrent statements are simpler for combinational logic.
6. Sequential statements are suitable for complex control logic.

14. What is the use of the process block in VHDL?

notes.arpanadhikari7.com.np

A process block in VHDL is used to describe **sequential behavior** within a concurrent environment.

Use of Process Block:

- Allows use of sequential statements like IF, CASE, and loops.
- Used to model combinational and sequential circuits.
- Sensitivity list determines when the process executes.
- Enables clock-based modeling for flip-flops and registers.

Significance:

- Helps in writing readable and structured code.
- Essential for behavioral modeling.
- Used to represent real-time behavior in digital systems.

Thus, the process block bridges sequential logic with concurrent hardware execution.

Available on notes.arpanadhikari7.com.np

15. Explain the significance of the wait statement in VHDL.

The `wait` statement controls the **execution timing** of a process in VHDL.

Significance of Wait Statement:

- It pauses the execution of a process until a condition is met.
- Can wait for signal changes, time delays, or conditions.
- Often used in testbenches and clock generation.

Types of Wait Conditions:

- Wait on signal
- Wait for time
- Wait until condition

Importance:

- Provides precise timing control.
- Improves simulation accuracy.
- Used when sensitivity lists are not preferred.

16. Describe the purpose of the if-else and case statements in VHDL with examples.

IF-ELSE and CASE statements are **conditional statements** used in behavioral modeling.

IF-ELSE Statement:

- Used for conditional decision-making.
- Evaluates conditions sequentially.
- Suitable for complex conditions.

notes.arpanadhikari7.com.np

Purpose:

- Used in combinational and sequential logic.
- Commonly used in control logic.

CASE Statement:

- Selects one choice among many based on an expression.
- All possible cases must be covered.
- More readable than multiple IF-ELSE statements.

Purpose:

- Used for state machines and decoders.
- Improves clarity and synthesis efficiency.

Both statements help implement decision-making logic in VHDL designs.

Available on notes.arpanadhikari7.com.np

17. How is a for loop used in VHDL? Write a code snippet to demonstrate it.

A FOR loop in VHDL is used to **repeat a set of statements** for a fixed number of iterations.

Use of FOR Loop:

- Simplifies repetitive operations.
- Used inside process blocks.
- Commonly used for array and vector operations.

Code Snippet Example:

```
process(A)
begin
  for i in 0 to 3 loop
    B(i) <= A(i);
  end loop;
end process;
```

Explanation:

- The loop copies values from vector A to vector B.
- The loop executes from index 0 to 3.
- Improves code compactness and readability.

18. What is testbench? Explain the role of test benches in VHDL.

A testbench is a **VHDL model used to test and verify** the functionality of a design.

What is a Testbench:

- It has no input or output ports.
- It applies test signals to the design under test (DUT).
- Observes and verifies output responses.

notes.arpanadhikari7.com.np

Role of Testbench in VHDL:

1. Verifies correctness of the design.
2. Detects errors before hardware implementation.
3. Simulates real-world operating conditions.
4. Helps in debugging and validation.
5. Improves design reliability.
6. Saves cost and development time.

Testbenches are essential for ensuring that a VHDL design works as expected before synthesis.

Available on notes.arpanadhikari7.com.np

Programs

1. 4-Bit Binary Adder

Entity

```
entity adder4 is
  port(A, B : in std_logic_vector(3 downto 0);
       Cin  : in std_logic;
       Sum  : out std_logic_vector(3 downto 0);
       Cout : out std_logic);
end adder4;
```

Data Flow Modeling

```
architecture dataflow of adder4 is
begin
  (Cout & Sum) <= ('0' & A) + ('0' & B) + Cin;
end dataflow;
```

Behavioral Modeling

```
architecture behavioral of adder4 is
begin
  process(A, B, Cin)
    variable temp : std_logic_vector(4 downto 0);
  begin
    temp := ('0' & A) + ('0' & B) + Cin;
    Sum  <= temp(3 downto 0);
    Cout <= temp(4);
  end process;
end behavioral;
```

Structural Modeling

```
architecture structural of adder4 is
begin
  (Cout & Sum) <= ('0' & A) + ('0' & B) + Cin;
end structural;
```

notes.arpanadhikari7.com.np

Testbench

```
entity tb_adder4 is
end tb_adder4;

architecture tb of tb_adder4 is
  signal A, B, Sum : std_logic_vector(3 downto 0);
  signal Cin, Cout : std_logic;
begin
  uut: entity work.adder4 port map(A, B, Cin, Sum, Cout);

  process
  begin
    A <= "0011"; B <= "0101"; Cin <= '0';
    wait for 10 ns;
    A <= "1111"; B <= "0001"; Cin <= '0';
    wait;
  end process;
end tb;
```


Available on notes.arpanadhikari7.com.np

Waveform Result:
Sum and Cout change according to binary addition of A, B, and Cin.

---

2. 4-Bit Binary Subtractor

```
entity sub4 is
  port(A, B : in std_logic_vector(3 downto 0);
       Diff : out std_logic_vector(3 downto 0));
end sub4;
```

Behavioral

```
architecture behavioral of sub4 is
begin
  process(A, B)
  begin
    Diff <= A - B;
  end process;
end behavioral;
```

Testbench

```
process
begin
  A <= "1000"; B <= "0011";
  wait for 10 ns;
  wait;
end process;
```

Waveform:
Diff shows subtraction result.

---

3. JK Flip-Flop

notes.arpanadhikari7.com.np

```
entity jkff is
  port(J, K, CLK : in std_logic;
       Q : out std_logic);
end jkff;
```

Behavioral

```
architecture behavioral of jkff is
  signal q_int : std_logic := '0';
begin
  process(CLK)
  begin
    if rising_edge(CLK) then
      if J='0' and K='0' then q_int <= q_int;
      elsif J='0' and K='1' then q_int <= '0';
      elsif J='1' and K='0' then q_int <= '1';
      else q_int <= not q_int;
      end if;
    end if;
  end process;
```


Available on notes.arpanadhikari7.com.np

```
  Q <= q_int;
end behavioral;
```

Waveform:
Q toggles based on J and K on clock edges.

---

4. Octal to Binary Converter

```
entity oct2bin is
  port(O : in std_logic_vector(2 downto 0);
       B : out std_logic_vector(2 downto 0));
end oct2bin;

architecture behavioral of oct2bin is
begin
  B <= O;
end behavioral;
```

Waveform:
Binary output equals octal input.

---

5. 3-Bit Counter

```
entity counter3 is
  port(CLK : in std_logic;
       Q : out std_logic_vector(2 downto 0));
end counter3;

architecture behavioral of counter3 is
  signal cnt : std_logic_vector(2 downto 0) := "000";
begin
  process(CLK)
  begin
    if rising_edge(CLK) then
      cnt <= cnt + 1;
    end if;
  end process;
  Q <= cnt;
end behavioral;
```

notes.arpanadhikari7.com.np

Waveform:
Q counts from 000 to 111 repeatedly.

---

6. 4-Bit Shift Left Register

```
entity shiftleft is
  port(CLK : in std_logic;
       A : in std_logic_vector(3 downto 0);
       Q : out std_logic_vector(3 downto 0));
end shiftleft;

architecture behavioral of shiftleft is
```


Available on notes.arpanadhikari7.com.np

```
begin
  process(CLK)
  begin
    if rising_edge(CLK) then
      Q <= A(2 downto 0) & '0';
    end if;
  end process;
end behavioral;
```

Waveform:
Bits shift left on each clock pulse.

---

7. Sequence Detector (1101)

Behavioral (Overlapping)

```
entity seq1101 is
  port(CLK, X : in std_logic;
       Z : out std_logic);
end seq1101;

architecture behavioral of seq1101 is
  type state is (S0,S1,S2,S3);
  signal ps, ns : state;
begin
  process(CLK)
  begin
    if rising_edge(CLK) then
      ps <= ns;
    end if;
  end process;

  process(ps, X)
  begin
    Z <= '0';
    case ps is
      when S0 => if X='1' then ns<=S1; else ns<=S0; end if;
      when S1 => if X='1' then ns<=S2; else ns<=S0; end if;
      when S2 => if X='0' then ns<=S3; else ns<=S2; end if;
      when S3 => if X='1' then Z<='1'; ns<=S1; else ns<=S0; end if;
    end case;
  end process;
end behavioral;
```

Waveform:
Z becomes 1 when sequence 1101 is detected.

---

8. 8×1 MUX and 1×8 DEMUX

8×1 MUX

```
Y <= D(to_integer(unsigned(S)));
```

1×8 DEMUX

Available on notes.arpanadhikari7.com.np

```
D <= (others=>'0');
D(to_integer(unsigned(S))) <= A;
```

---

9. 8-to-3 Encoder and 3-to-8 Decoder

Encoder

```
process(D)
begin
  case D is
    when "00000001" => Y <= "000";
    when "00000010" => Y <= "001";
    when "00000100" => Y <= "010";
    when others => Y <= "000";
  end case;
end process;
```

Decoder

```
D <= "00000001" sll to_integer(unsigned(A));
```

---

10. Full Adder Using Two Half Adders

```
Sum  <= A xor B xor Cin;
Cout <= (A and B) or (Cin and (A xor B));
```

---

11. BCD Counter

```
entity bcd is
  port(CLK : in std_logic;
       Q : out std_logic_vector(3 downto 0));
end bcd;

architecture behavioral of bcd is
  signal cnt : std_logic_vector(3 downto 0) := "0000";
begin
  process(CLK)
  begin
    if rising_edge(CLK) then
      if cnt = "1001" then
        cnt <= "0000";
      else
        cnt <= cnt + 1;
      end if;
    end if;
  end process;
  Q <= cnt;
end behavioral;
```

notes.arpanadhikari7.com.np

Waveform:
Counts from 0000 to 1001 and resets.

Available on notes.arpanadhikari7.com.np